

03-语义分析与中间代码复习讲义

目录

1	03 语义分析与中间代码复习讲义
---	------------------

2

1 03 语义分析与中间代码复习讲义

对应资料:

- Slides/Slides/Ch6-1-Intermediate Code Generation.pdf
- Slides/Slides/Ch6-2-Translation into SSA.pdf
- Slides/Slides/Slides-Ant/7-symtable-handout.pdf
- Slides/Slides/Slides-Ant/8-type-systems-out.pdf
- Slides/Slides/Slides-Ant/9-semantics-ag-handout.pdf
- Slides/Slides/Slides-Ant/10-llvm-handout.pdf
- Slides/Slides/Slides-Ant/11-ir-expr-control-handout.pdf
- Review/compile.pdf
- Exam/2025.pdf 支配问题
- Exam/过时/2022-exam-A.pdf 题目 5、6
- Exam/过时/2023-exam-A.pdf 题目 4、5

1.1 一、语义分析

语义分析在语法结构上检查程序含义，常见任务:

- 类型检查。
- 作用域和符号表维护。
- 声明与使用匹配。
- 函数调用参数个数和类型检查。
- 语义错误报告。

典型数据结构:

```
SymbolTable
  enterScope()
  exitScope()
  define(name, type, attributes)
  lookup(name)
```

1.2 二、语法制导定义

语法制导定义把属性和语义规则附加在产生式上。

两类属性：

- 综合属性：由子节点向父节点传递。
- 继承属性：由父节点或兄弟节点向当前节点传递。

答题格式：

产生式	语义规则
$S \rightarrow L \ . \ L$	$S.val = L1.val + L2.val * pow(2, -L2.len)$
$L \rightarrow L \ B$	$L.val = L1.val * 2 + B.val; L.len = L1.len + 1$
$B \rightarrow 0$	$B.val = 0$
$B \rightarrow 1$	$B.val = 1$

1.3 三、语法制导翻译常见题

1.3.1 统计声明个数

例： $P \rightarrow D, D \rightarrow D ; D \mid T \ id \mid func \ id \{D\}$ 。

可定义综合属性：

$D.count$: D 内声明的 id 总数
 $P.count$: 程序总声明数

语义规则：

$P \rightarrow D$	$P.count = D.count$
$D \rightarrow D1 ; D2$	$D.count = D1.count + D2.count$
$D \rightarrow T \ id$	$D.count = 1$
$D \rightarrow func \ id \{D1\}$	$D.count = 1 + D1.count$

1.3.2 嵌套深度

可定义继承属性：

$D.depth$: 当前作用域深度

语义规则：

$P \rightarrow D$	$D.depth = 0$
$D \rightarrow D1 ; D2$	$D1.depth = D.depth; D2.depth = D.depth$
$D \rightarrow T \ id$	$print(id, D.depth)$
$D \rightarrow func \ id \{D1\}$	$D1.depth = D.depth + 1$

1.4 四、三地址码

三地址码每条指令最多包含三个地址。

常见形式：

```
x = y op z
x = op y
x = y
goto L
if x relop y goto L1
if x relop y goto L1 else goto L2
param x
call f, n
return x
```

1.5 五、回填

回填用于布尔表达式和控制流翻译。核心是先留下跳转目标空位，之后再填标签。

常用属性：

```
B.truelist
B.falselist
S.nextlist
```

常用函数：

```
makelist(i)    : 创建只含第 i 条指令的链表
merge(p1,p2)   : 合并两个链表
backpatch(p,L): 把链表 p 中的跳转目标填为 L
nextquad       : 下一条中间代码编号
```

布尔表达式：

```
B -> E1 relop E2
  B.truelist = makelist(nextquad)
  emit("if E1 relop E2 goto _")
  B.falselist = makelist(nextquad)
  emit("goto _")
```

while 语句：

```
S -> while M1 B do M2 S1
  backpatch(B.truelist, M2.quad)
  backpatch(S1.nextlist, M1.quad)
  emit("goto M1.quad")
  S.nextlist = B.falselist
```

1.6 六、SSA

SSA: Static Single Assignment。每个变量只有一个定义。

例:

```
x1 = 1
if c goto L1 else L2
L1: x2 = 2
L2: x3 = phi(x1, x2)
```

优点:

- def-use 链清楚。
- 便于常量传播、死代码删除、数据流分析。
- 每个使用点能定位定义来源。

1.7 七、支配关系

支配:

$B \text{ dominates } A$

含义: 从入口到 A 的每条路径都经过 B。

后支配:

$B \text{ post-dominates } A$

含义: 从 A 到出口的每条路径都经过 B。

直接支配:

$\text{idom}(A)$

含义: 严格支配 A 且离 A 最近的支配者。

性质:

- 基本块的直接支配关系构成一棵以入口为根的树。
- 每个非入口节点有且只有一个直接支配者。
- 支配树从根到某节点的路径就是该节点全部支配者。

1.8 八、支配边界与 phi 插入

支配边界：

$$DF(B) = \{A \mid B \text{ 支配 } A \text{ 的某个前驱, 且 } B \text{ 不严格支配 } A\}$$

直观理解：刚离开 B 支配范围的位置。

phi 插入：

1. 找出变量 x 的所有定义块集合 $Def(x)$ 。
2. 计算 $Def(x)$ 的迭代支配边界。
3. 在这些块插入 $x = phi(\dots)$ 。
4. 对变量重新命名，形成 SSA。

1.9 九、自测题与答案

1.9.1 题 1：综合属性和继承属性区别。

答案：综合属性从子节点向父节点计算；继承属性从父节点或兄弟节点向当前节点传递。

1.9.2 题 2：回填解决什么问题？

答案：生成布尔表达式和控制流代码时，跳转目标尚未确定。回填先记录待填跳转指令，等目标标签确定后统一填入。

1.9.3 题 3：什么位置要插入 phi 函数？

答案：变量多个定义流向汇合处要插入 phi；算法上在变量定义块集合的迭代支配边界插入。